

Study Guide

A self-paced, offline curriculum: master the reference engine one week at a time, then port each lesson directly into your own M151 Engine — file by file.

Companion to: `examples/simple_engine/`

Target project: the M151_Engine repo root (`core/`, `gameplay/`, `utils/`, `games/`)

Pace: four weekly sessions, each self-contained — repeat any week freely

No internet required: every answer, formula, and fix is inline

Table of Contents

0	How To Use This Guide Offline	Read this first
1	Week 1 — Time, Space, and the Camera	dt, world/screen space, camera
2	Week 2 — Entities, Managers, the Game Loop	the missing architecture
3	Week 3 — Animation, Collision, Input	filling in the empty stubs
4	Week 4 — Data-Driven Design & Full Port	the capstone
5	Glossary	every term, defined plainly
6	Offline Debugging Cookbook	error → cause → fix, no internet needed
7	Self-Assessment Checklist	can you, without looking?
8	Cheat Sheet	every formula, one page
9	Migration Map	simple_engine file → your file → action

0 · How To Use This Guide Offline

This guide is built for the next few weeks with no internet — no Stack Overflow, no docs site, no asking Claude mid-thought. That changes how you should study: instead of reading passively, treat every section as a loop of four steps.

- 1 **Open the file.** Each section names exact files under `engine/`, `entities/`, or `scenes/`. Open them in an editor before reading the explanation — the guide is a companion to the code, not a replacement for reading it.
- 2 **Read the Concept.** One idea at a time.
- 3 **Answer "Check Yourself" before reading the answer.** Cover the answer with your hand or a piece of paper. Getting it wrong and then reading why is worth more than reading it once.
- 4 **Do the Drill, then the Port-It task.** The Drill happens inside `examples/simple_engine/` — safe to break. The Port-It task happens in your real `M151_Engine` project — this is where the studying turns into actual progress on your engine.

WHY OFFLINE CHANGES YOUR DEBUGGING HABITS

Normally, an unfamiliar traceback gets pasted into a search bar. You can't do that for a few weeks, so lean on three things instead: (1) read the traceback from the **bottom up** — the last line is the actual error, everything above it is just the chain of calls that led there; (2) add `print()` statements liberally and remove them later — it's not "cheating," it's how most engine bugs actually get found even with internet access; (3) Section 6 of this guide is a pre-loaded answer key for the specific errors this kind of code tends to produce. Check there before assuming you're stuck.

KEEP A NOTES FILE

Create `NOTES.md` at the root of `M151_Engine` and jot a dated line every time something breaks and you fix it, or something clicks. Future you — and any future conversation with Claude once you're back online — benefits enormously from a real record of what you actually learned, versus trying to reconstruct it from memory weeks later.

Don't rush. Each "week" assumes a handful of sessions, not one sitting. If a week doesn't stick, repeat it before moving on — the weeks build on each other, and Week 4's capstone assumes Weeks 1-3 are solid.

1 · Time, Space, and the Camera

Open alongside this section: `engine/config.py` · `engine/camera.py` · `entities/player.py`

1.1 — Delta time (dt)

Concept. Every `update()` multiplies movement by `dt`, the seconds elapsed since the previous frame (`Game.run()` gets it from `clock.tick(FPS) / 1000.0`). A speed of 220 means "220 world pixels per second," full stop — regardless of whether the game runs at 30fps, 60fps, or 240fps.

CHECK YOURSELF

If `PLAYER_SPEED = 220` and the game runs at a steady 50fps, roughly how many pixels does the player move in one frame? (*Cover this line before reading on.*)

Answer: $dt \approx 1/50 = 0.02s$. $220 \times 0.02 = 4.4$ pixels per frame. At 60fps it'd be $220 \times 0.0167 \approx 3.7$ px/frame — different per-frame step, same real-world speed.

DRILL

In `entities/player.py`, temporarily hardcode `self.world_x += 1` (no `dt`, no speed) inside `update()`. Run the demo and watch the tank's speed change if you resize your terminal load or otherwise stress the machine (frame rate drops). That visible inconsistency is exactly what `dt` exists to prevent. Revert the change afterward.

1.2 — Three coordinate spaces

Concept. World space is where an entity actually lives on the full map (the demo's map is 1920×5760). Camera space is the 960×540 slice of that map currently visible. Screen space is pixels from the window's top-left corner, always in `[0, 960) × [0, 540)` for anything on-screen. `Camera.world_to_screen()` is the only bridge between world space and screen space:

```
screen_x = world_x - camera.x
screen_y = world_y - camera.y
```

CHECK YOURSELF

The player is at world (2100, 3080). The camera's top-left is at world (1632, 2810). Where does the player get drawn on screen?

Answer: `screen_x = 2100 - 1632 = 468`. `screen_y = 3080 - 2810 = 270`. Drawn at pixel (468, 270).

WHY ENTITY.DRAW() NEVER TOUCHES CAMERA.X/CAMERA.Y

Every entity's `draw(surface, camera)` calls `camera.world_to_screen(...)` itself and blits at the result — it never reaches into `camera.x` directly. That keeps the translation logic in exactly one place, so if you ever change how the camera computes its position (e.g. adding screen shake), every entity's draw call keeps working unmodified.

1.3 — The camera's dead zone and edge clamp

Concept. `Camera.follow()` doesn't lock onto the player's exact center every frame (that causes jitter on small movements) — it only re-aims once the player drifts past a threshold (middle-third horizontally, roughly half-to-four-fifths vertically), then clamps so the camera can never show past the map edge.

CHECK YOURSELF

Near the right edge of the map, does the player's on-screen X position stay fixed at the center of the screen, or drift toward the right edge of the screen?

Answer: It drifts toward the right edge. The camera's `self.x` gets clamped at `map_w - screen_w` and stops moving, but the player keeps walking — so their screen position (which depends on both) keeps increasing until they hit the map's actual edge.

DRILL

Run the demo, walk to each of the four map corners, and watch the HUD / tank position. Predict, before you get there, whether the tank will sit dead-center or slide toward a screen edge at each corner — then verify.

A REAL BUG IN YOUR PROJECT, RIGHT NOW

Open `games/main-simple.py` line 41: `self.player = Entity(world_x=..., world_y=..., frames=...)`. Compare it against `Entity.__init__`'s signature in `core/entities/entity.py`, which requires `id` as the first parameter with no default value. Before running it — what error do you expect? Then run `games/main-simple.py` and confirm. (Fixing it is a Week 2 Port-It task — for now, just diagnose it.)

2 · Entities, Managers, and the Game Loop

Open alongside this section: `engine/entity.py` · `engine/entity_manager.py` · `engine/scene.py` · `engine/game.py` · `entities/missile.py`

2.1 — One shape for everything

Concept. `Entity` is the base class for *everything* that lives in the world: position, size, an `alive` flag, `update()`, `draw()`, `get_hitbox()`. `Player`, `Enemy`, and `Missile` all inherit from it here.

CHECK YOURSELF

Your main project's `core/entities/missile.py` — does `Missile` inherit from `Entity`? Open the file and check the class declaration.

Answer: No — `class Missile:` with no base class, even though it has `world_x`/etc. equivalents (`self.x`, `self.y`) and matching `update()`/`draw()`/`get_hitbox()` methods. This is exactly the inconsistency that forced every prototype to hand-write its own missile-vs-enemy collision check instead of a reusable one.

2.2 — EntityManager: one registry, named groups

Concept. Instead of `self.enemy` (one hardcoded slot) or `self.missiles` (a bare list), every entity goes into `EntityManager` under a named group: `entities.add(obj, group="enemies")`. `Update`, `draw`, and `cleanup` become one loop each, regardless of how many groups exist.

WHY KILL() IS DEFERRED

`Entity.kill()` only sets `alive = False`. Removal happens once per frame, in `EntityManager.purge_dead()`, *after* collision checks finish.

CHECK YOURSELF

Why does removing an item from a list *while looping over that same list* cause bugs? Work through this by hand: a list `[A, B, C]`, you're at index 0 (A), you call `list.remove(A)`. What's the list now, and what does the loop visit next?

Answer: The list becomes `[B, C]`. The loop's internal index advances to 1, which is now `C` — `B` got silently skipped. In a collision loop, that's an enemy that "survives" a hit purely by chance of ordering.

2.3 — Scene and Game: the loop, written once

Concept. `Scene` is three empty hook methods (`handle_event`, `update`, `draw`). `Game` owns the window and clock, and calls whichever `Scene` is currently active. Swapping scenes is the entire mechanism for changing game state.

A BUG THIS PROJECT ACTUALLY HIT

```
game = Game()          # pygame.init() + set_mode() happen here
game.run(PlayScene())  # PlayScene is only constructed AFTER that
```

Check yourself: why can't `Game` just take a `Scene` directly in its constructor, as in `Game(PlayScene())`?

Answer: Python evaluates arguments before making the call — `PlayScene()` would run first, and its constructor loads images via `Surface.convert()`, which requires a display mode to already exist.

`Game.__init__` is what calls `pygame.display.set_mode()`, but it wouldn't get the chance to run until *after* `PlayScene()` already crashed. This is a real ordering bug that came up while building this very demo.

Port-It — Week 2 tasks for your M151_Engine

- 1 Fix `core/entities/player.py`: its `__init__` takes `(self, world_x, world_y, frames)` and calls `super().__init__(world_x, world_y, frames)` — but `Entity.__init__` needs `id` first. Add an `entity_id` parameter and pass it through.
- 2 Give `core/entities/entity.py` an `alive` flag and a `kill()` method, matching `engine/entity.py` here.
- 3 Make `core/entities/missile.py`'s `Missile` inherit from `Entity`, following `entities/missile.py` in this demo as the template.
- 4 Create a new `core/systems/entity_manager.py` — there is no equivalent in your project yet. Port `engine/entity_manager.py` in directly; it doesn't depend on anything game-specific.
- 5 Create `core/scene.py` and `core/game.py` (also missing), and refactor `games/main-with_enemy.py`'s hand-rolled loop into a `Scene` subclass that plugs into them. This is the biggest single upgrade you can make to the project — it's what turns three copy-pasted prototypes into one reusable structure.

3 · Animation, Collision, Input

Open alongside this section: `engine/animation.py` · `engine/collision.py` · `engine/input.py`

3.1 — Animator

Concept. `Animator` stores `{direction: [Surface, ...]}` — even a single `Surface` gets normalized into a one-item list. `update()` only advances the frame index when a direction has more than one frame; a direction with exactly one frame never animates. This is why the demo, whose art only has one frame per direction, still uses a "real" animation system instead of a simpler direction-only lookup — it's already shaped to accept a walk-cycle sheet without any caller-side changes.

CHECK YOURSELF

If you passed `Animator({"right": [frame_a, frame_b, frame_c]}, fps=6)`, roughly how often does the displayed frame change?

Answer: `fps=6` means `seconds_per_frame = 1/6 ≈ 0.167s`. Every `~167ms`, `_frame_index` advances by one, wrapping back to 0 after `frame_c`.

3.2 — Collision

Concept. `collision.check_groups(a, b, on_hit)` builds one list of hitboxes for group B, then checks every entity in group A against that list with `pygame.Rect.collidelist()` — an $O(n)$ scan. Hitboxes always come from `Entity.get_hitbox()`, in world space, never screen space.

CHECK YOURSELF

Why must collision be computed in world space rather than screen space?

Answer: Screen-space coordinates depend on where the camera currently is. Two entities could be perfectly overlapped in the world but drawn at different screen offsets mid-frame if you checked screen rects computed at different points in the update — world space is the single source of truth that doesn't shift under you.

WHY NO SPATIAL GRID / QUADTREE

$O(n)$ per query is fine at the entity counts a learning project deals with — tens, not thousands. Building a spatial partitioning structure before you can measure it being too slow is solving a problem you don't have. Revisit this only once profiling (not guessing) tells you collision is the bottleneck.

3.3 — Input, revisited

Concept recap. Diagonal movement gets multiplied by $1/\sqrt{2} ≈ 0.707$ so its vector length matches a cardinal move's length of 1.0 — otherwise diagonal movement is `~41%` faster, a side effect of the Pythagorean theorem. (Full derivation in the usage guide, Chapter 4, if you kept a copy.)

Port-It — Week 3 tasks for your M151_Engine

- 1 `core/systems/animation.py` is currently an **empty, 0-byte file** in your project. Port `engine/animation.py` into it directly.
- 2 `core/systems/collision.py` is also **empty**. Port `engine/collision.py` in, then replace the repeated `m.get_hitbox(...).collidelist(...)` blocks duplicated across `games/main-with_enemy.py` and `games/01_Spawner/main-with_spawner.py` with one call to `collision.check_groups()`.
- 3 `utils/resource_loader.py` is also **empty** — port `engine/resource_loader.py`'s cache-and-slice pattern in, consolidating what `gameplay/graphics/graphic_loader.py` and the older `gameplay/actions/unit_actions/main_character_actions.py` each do separately.

- 4 `utils/math_utils.py` is also **empty** — a good home for the diagonal-normalization constant and the sheet cell-position formula, both currently duplicated as raw numbers across several files.

4 · Data-Driven Design & Full Port

Open alongside this section: `engine/spawner.py` · `data/enemies.json` · `scenes/play_scene.py`

4.1 — The factory-injection pattern

Concept. `JSONSpawner` doesn't know how to build an `Enemy` — it just reads JSON and calls whatever `factory` function you hand it, once per spec. The factory (`PlayScene._make_enemy`) is what actually knows about `Enemy`. This indirection means the exact same `JSONSpawner` class could spawn pickups, obstacles, or anything else, from a totally different JSON file, with zero changes to `JSONSpawner` itself.

CHECK YOURSELF

If you wanted to add a "pickup" spawner using the same `JSONSpawner` class, what would you actually need to write?

Answer: Just a new factory function — e.g. `make_pickup(spec, world_x, world_y) -> Pickup` — and a call like `spawner.load("pickups", make_pickup)`. Nothing in `JSONSpawner` changes.

4.2 — Offsets from center, not absolute pixels

Concept. `enemies.json` stores `spawn_x_from_center` / `spawn_y_from_center` rather than absolute coordinates. `JSONSpawner.load()` adds `map_w / 2` to get the real position. Swap in a differently-sized map and the same file still places enemies sensibly.

Port-It — Week 4 tasks (the capstone)

- 1 `gameplay/spawner/spawner.py` is currently **empty**. Port `engine/spawner.py`'s `JSONSpawner` in directly — it has no dependency on game-specific code, same as every other `engine/` file.
- 2 `gameplay/level/level_loader.py` **and** `gameplay/level/level1.json` are both empty — this is your green field. Design your own schema (map image path, plus a spawn list in the same offset-from-center shape as `enemies.json`), and write a `LevelLoader` that reads it the way `PlayScene.__init__` reads `MAP_IMAGE` and `ENEMIES_JSON` here.
- 3 **Capstone:** once Weeks 2–4's ports are done, rebuild one of `games/main-with_enemy.py` or `games/01_Spawner/main-with_spawner.py` using *only* the ported `core/` and `gameplay/` systems — no copy-pasted loop logic, no hand-rolled enemy list. If the new version is dramatically shorter than the original, the port worked. If it isn't, that's useful information too: it tells you which piece didn't actually get consolidated, and that becomes next session's task.

5 · Glossary

Delta time (dt)

Seconds elapsed since the previous frame. Multiplying a speed by dt converts "pixels per second" into "pixels this frame," keeping motion consistent regardless of frame rate.

World space

The coordinate system of the full map — an entity's "true" position, independent of what the camera is currently showing.

Screen space

Pixel coordinates relative to the window's top-left corner — where something is actually drawn. Always derived from world space via the camera, never stored directly on an entity.

Camera dead zone

A margin in which the camera doesn't re-center on the player every frame, preventing jitter from small movements. The camera only moves once the player crosses the zone's edge.

AABB (Axis-Aligned Bounding Box)

A rectangle that isn't rotated relative to the screen — the simplest possible collision shape, and what `pygame.Rect` represents.

Hitbox

The rectangle (usually an AABB) used to test whether one entity overlaps another, which may be smaller than the entity's full sprite.

Entity

Anything that exists in the game world with a position and a lifecycle (created, updated every frame, eventually destroyed).

Composition over inheritance

Building behavior by giving an object other objects to delegate to (e.g. `Player` *has an* `Animator`) rather than by deep inheritance chains. This engine uses both: entities inherit from `Entity` for shared shape, but compose an `Animator` for sprite behavior.

Deferred deletion / kill queue

Marking an object dead (`alive = False`) instead of removing it immediately, then sweeping all dead objects out in one pass after every system has had a chance to see them this frame. Avoids mutate-while-iterating bugs.

Scene

One screen or mode of the game (playing, paused, title, game-over). Swapping which Scene is active is how the game changes state.

Game loop

The while running: `handle input` → `update` → `draw` cycle that runs once per frame for the entire life of the program.

Factory pattern

A function whose job is "build one of these," passed as a parameter so the caller doesn't need to know the concrete type it's building. Used here so `JSONSpawner` can build any kind of entity without importing it.

Data-driven design

Putting tunable content (enemy positions, speeds, trails) in a data file like JSON instead of hardcoding it in source, so changing the game doesn't require changing code.

Colorkey

A single RGB color treated as fully transparent when blitting — a cheap alternative to a real alpha channel, common in older sprite sheets (this project's colorkey is cyan, `(32, 200, 248)`).

convert() vs convert_alpha()

`convert()` matches a Surface's pixel format to the display for fast blitting, with no per-pixel transparency. `convert_alpha()` does the same but preserves a real alpha channel. Both require a display mode to already exist (`pygame.display.set_mode()`), which is why `Game` must be constructed before any `Scene` that loads images.

Sprite sheet

A single image file containing many small sprite frames arranged in a grid, cut apart at load time by row/column math instead of shipping one file per frame.

Vector normalization

Scaling a vector so its length becomes exactly 1.0, without changing its direction — used here so diagonal movement isn't faster than cardinal movement.

Spatial partitioning

Dividing the world into a grid or tree so collision checks only compare nearby entities instead of everything against everything. Not needed at this project's entity count — see 3.2.

Tilemap

A level represented as a grid of small reusable tile images plus a 2D array of tile indices, instead of one large pre-rendered map image. Not used in this demo (see Chapter 9 of the usage guide).

6 • Offline Debugging Cookbook

Error message (or symptom) → likely cause → fix. Check here before assuming you're stuck.

Symptom / error	Cause	Fix
<code>pygame.error:</code> cannot convert without <code>pygame.display</code> initialized	An image got loaded (<code>.convert()</code> / <code>.convert_alpha()</code>) before <code>pygame.display.set_mode()</code> ran — usually because a Scene was constructed before the Game that owns the window.	Build <code>Game()</code> first (as its own statement), then pass a freshly-constructed Scene to <code>game.run(...)</code> . See Week 2.3.
<code>TypeError:</code> <code>__init__()</code> missing 1 required positional argument: 'id'	A subclass's <code>__init__</code> forgot to accept and forward an <code>id/entity_id</code> to <code>Entity.__init__</code> .	Add the missing parameter and pass it through <code>super().__init__(...)</code> . This exact bug exists in <code>core/entities/player.py</code> right now — see Week 1.3.
<code>KeyError:</code> 'right' (or any direction string) indexing a frames dict	A direction string produced by input/movement code doesn't exactly match a key in the sheet's coordinate map — often a typo, or mixing "right" vs <code>RIGHT</code> constant vs a different casing.	<code>print(sorted(frames.keys()))</code> next to the failing lookup and compare against the direction constants in <code>config.py</code> .
A hit enemy sometimes doesn't disappear	Removing from a list while iterating over that same list — the element after the removed one gets silently skipped.	Mark dead with a flag, sweep in a separate pass afterward (deferred deletion — see Week 2.2). Never call <code>list.remove()</code> inside a <code>for</code> loop over that list.
Diagonal movement feels noticeably faster	Diagonal input isn't scaled by $1/\sqrt{2} \approx 0.707$, so its vector length is ~ 1.41 instead of 1.0.	Multiply both diagonal components by 0.707 before applying speed and dt. See Week 1 / Week 3.3.
Camera jitters or vibrates during small movements	The camera re-centers on the player's exact position every single frame with no tolerance.	Add a dead zone: only re-aim once the player crosses a threshold distance from the camera's current aim point. See Week 1.3.
Sprite shows the wrong cell, or bleeds into a neighboring one	Off-by-one in row/col-to-pixel math, or forgetting the 1px separator line between cells in the sheet's pitch (33px per cell, not 32px).	Re-derive <code>x = col_start_x + col * pitch</code> by hand for one known-good cell and compare against what the code computes.
Sprite shows as a solid colored box instead of transparent	Forgot <code>set_colorkey(...)</code> after slicing, or called <code>.convert()</code> on a sheet that actually needs real alpha (<code>.convert_alpha()</code>).	Match the loading method to the art: <code>colorkey</code> sheets need <code>set_colorkey</code> after every slice; real-alpha PNGs need <code>convert_alpha()</code> , not <code>convert()</code> .
Window opens and closes instantly, or never draws anything	Missing <code>pygame.display.flip()</code> at the end of the frame, or the event loop isn't reached because of an earlier crash swallowed by a broad <code>except</code> .	Confirm <code>flip()</code> runs every frame; avoid bare <code>except:</code> blocks that hide the real traceback.
Everything moves at wildly different speeds on different runs	A speed value is being applied without multiplying by <code>dt</code> .	Every per-frame movement should look like <code>position += velocity * dt</code> , never <code>position</code> <code>+= velocity</code> alone.

7 · Self-Assessment Checklist

Work through each week's list from memory, without opening the code. If more than one or two feel shaky, revisit that week before moving on.

Week 1 — Time, Space, Camera

- ☐ I can explain why multiplying by `dt` keeps speed frame-rate independent.
- ☐ I can name all three coordinate spaces and which one an entity's position is stored in.
- ☐ I can write `world_to_screen` from memory.
- ☐ I can explain why the camera has a dead zone instead of centering every frame.

Week 2 — Entities, Managers, Game Loop

- ☐ I can explain why every game object should inherit from one common Entity class.
- ☐ I can explain why `kill()` only sets a flag instead of removing immediately.
- ☐ I can explain why `Game()` must be constructed before any Scene that loads images.
- ☐ I know exactly which files in my own project still need this architecture ported in.

Week 3 — Animation, Collision, Input

- ☐ I can explain why Animator normalizes single Surfaces into one-item lists.
- ☐ I can explain why collision hitboxes must be computed in world space, not screen space.
- ☐ I can explain why this project doesn't use a spatial grid for collision yet.
- ☐ I have located (and ideally already filled in) the empty stub files in my own project.

Week 4 — Data-Driven Design

- ☐ I can explain what the factory function passed to `JSONSpawner` is responsible for.
- ☐ I can explain why spawn positions are stored as offsets from map-center.
- ☐ I have a working `LevelLoader` in my own project, even a minimal one.
- ☐ I have rebuilt at least one of my own prototypes on top of the ported systems.

8 · Cheat Sheet

Every formula and pattern from this guide, in one place.

Delta-time movement

```
self.world_x += vx * speed * dt    # speed is "world px / second"
```

World → screen

```
screen_x = world_x - camera.x
screen_y = world_y - camera.y
```

Diagonal normalization

```
DIAG = 0.70710678 # 1/sqrt(2)
# cardinal:  dx, dy = 0, -1      (length 1.0)
# diagonal:  dx, dy = DIAG, -DIAG (length 1.0, not 1.414)
```

Camera edge clamp

```
self.x = max(0.0, min(target_x, float(map_w - screen_w)))
self.y = max(0.0, min(target_y, float(map_h - screen_h)))
```

Sprite-sheet cell position

```
x = col_start_x + col * cell_pitch    # cell_pitch = cell_size + separator_px
y = row_start_y + row * cell_pitch
```

Deferred kill / purge

```
entity.alive = False                # mark, don't remove
...
group[:] = [e for e in group if e.alive]    # sweep once, after all systems ran
```

Group-vs-group collision

```
b_boxes = [b.get_hitbox() for b in group_b]
for a in group_a:
    hit = a.get_hitbox().collidelist(b_boxes)
    if hit != -1:
        on_hit(a, group_b[hit])
```

Factory-injected spawning

```
def load(self, key, factory):
    for spec in data[key]:
        x = map_w/2 + spec["spawn_x_from_center"]
        y = map_h/2 + spec["spawn_y_from_center"]
        yield factory(spec, x, y)    # factory knows the concrete type; loader doesn't
```

Correct startup order

```
game = Game()                # pygame.init() + set_mode() first
game.run(PlayScene())        # THEN construct anything that loads images
```

9 • Migration Map

Every file in `examples/simple_engine/`, matched against its counterpart in your main `M151_Engine` project, with its current status and the action this guide recommends. This table is the single most useful reference for turning four weeks of study into real progress on your own engine.

simple_engine file	M151_Engine equivalent	Status	Action
<code>engine/entity.py</code>	<code>core/entities/entity.py</code>	Exists, missing <code>alive / kill()</code>	Add deferred-kill support
<code>engine/entity_manager.py</code>	<i>none</i>	Missing entirely	Create <code>core/systems/entity_manager.py</code>
<code>engine/camera.py</code>	<code>core/camera/camera.py</code>	Exists, already close	Confirm parity, keep as-is
<code>engine/animation.py</code>	<code>core/systems/animation.py</code>	Empty (0 bytes)	Port directly
<code>engine/collision.py</code>	<code>core/systems/collision.py</code>	Empty (0 bytes)	Port directly, remove duplicated inline checks
<code>engine/resource_loader.py</code>	<code>utils/resource_loader.py</code>	Empty (0 bytes)	Port directly, consolidate with <code>graphic_loader.py</code>
<code>engine/input.py</code>	<code>utils/keyboard_controller.py</code>	Exists (older API shape)	Merge diagonal-normalization logic; keep one version
<code>engine/spawner.py</code>	<code>gameplay/spawner/spawner.py</code>	Empty (0 bytes)	Port directly
<code>engine/scene.py</code> + <code>game.py</code>	<i>none</i>	Missing entirely	Create <code>core/scene.py</code> , <code>core/game.py</code>
<code>entities/player.py</code>	<code>core/entities/player.py</code>	Exists, broken (missing <code>id</code> arg)	Fix constructor — see Week 2
<code>entities/missile.py</code>	<code>core/entities/missile.py</code>	Exists, not an <code>Entity</code>	Refactor to inherit <code>Entity</code>
<code>entities/enemy.py</code>	<code>core/entities/enemy.py</code>	Exists, already close	Minor unification once <code>Entity</code> gains <code>kill()</code>
<code>data/enemies.json</code>	<code>games/01_Spawner/enemies.json</code>	Exists, same shape	Reuse format for <code>LevelLoader</code>
<i>(new concept)</i>	<code>gameplay/level/level_loader.py</code> + <code>level1.json</code>	Both empty	Greenfield — design your own schema, Week 4 capstone

M151 Simple Engine Study Guide — every lesson here points at a real file in your project. When you're back online, the fastest way to pick this back up is: open `NOTES.md`, check the Self-Assessment Checklist, and continue the Migration Map from wherever it stopped.